Teamwork and Reflection

Teamwork and Reflection: From Design to Implementation

During Assignment 1, our team designed the app mainly from the user's point of view: civilians needed to request help quickly, and responders needed enough information to make a decision under pressure. When we moved into Assignment 2, the biggest change was that these ideas had to become working Power Apps screens connected to SharePoint data. This made some of our earlier design assumptions much more concrete.

One of the main lessons was that simple UI decisions are only useful if the backend logic supports them clearly. For example, in the design stage we treated incident status as a display label. During implementation, we realised that status actually controlled the whole incident lifecycle: whether an incident appeared in the queue, whether it opened the detail screen or tracking screen, and whether it should be removed after being marked as Done or False Report. Because of this, we added a clearer status model using New, On the Way, Done, and False Report, and kept Support Requested as a separate Yes/No field instead of creating another status. This made the system easier to test and easier to explain.

Another lesson was that Power Apps works best when the workflow is kept focused. At first, we considered adding more options and more manual controls, but implementation showed that too many choices would make the app harder to use and harder to test. We reduced the civilian flow to the essential steps: choose role, send help, confirm incident type and location, then view status and safety guidance. For responders, we focused on the main operational actions: review the queue, open the incident, accept/dispatch, request support, flag false report, and mark as done. This helped us keep the prototype aligned with the original requirements instead of building extra features that were not central to the emergency response scenario.

We also learnt that testing can change how you understand your own design. Some behaviours looked correct on the screen but needed backend evidence to prove that they were actually working. For example, the support request warning was not enough by itself; we also checked that the same incident record in SharePoint changed to Support Requested = true and Status = On the Way. Similarly, marking an incident as Done or False Report had to be checked by confirming that it was removed from the active queue. This made our testing more meaningful because it checked the app behaviour, the screen transition, and the data update together.

A practical challenge was dealing with tool limitations. Power Apps Test Studio was useful for repeatable checks, but it did not reliably replay every control, especially the incident type

dropdown and some fast screen transitions. Instead of hiding this issue, we documented the limitation and used a workaround: stable parts were tested with Test Studio, while unstable parts were supported with manual screenshots, SharePoint backend evidence, and Monitor results. This was a useful lesson for future developers: automated tests are important, but they should be supported by manual and backend checks when the tool cannot capture the full behaviour.

Teamwork also changed during implementation. In the design stage, tasks could be separated more easily, such as UI design, requirements, diagrams, and report writing. During implementation, the work became more connected. A small change in Power Apps, such as renaming a control or changing the status logic, affected the diagrams, testing table, backend evidence, and user guide. To manage this, we used Jira and regular team communication to reassign tasks when the app changed. This helped us keep the documentation consistent with the final prototype rather than leaving old design decisions in the report.

The most transferable lesson from this project is that a prototype is not just a set of screens. It needs a clear data model, consistent status logic, realistic test evidence, and documentation that explains the system from both user and developer perspectives. If future developers continued this project, we would recommend first checking the status lifecycle and SharePoint field meanings before changing the UI. Most issues in this project were not caused by one broken button, but by the connection between screen behaviour, backend values, and test expectations.

Overall, the project taught us to move from “does this screen look right?” to “does this workflow actually work from user action to backend update?”. That shift was the most important part of moving from design to implementation.